



WEEK 9

-

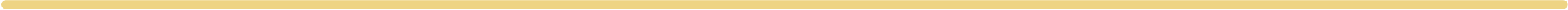
OBSERVABILITY

10 November 2025

11 November 2025

12 November 2025

Simone Maggiani



ABOUT ME

Name:

- Simone Maggiani

Education:

- Bachelor's Degree in Computer Engineering

Past Working Experiences:

- Senior Consultant in Capgemini (11+ years)

Current Working Experience:

- **Solution Architect** in MSC Technology (4+ years)
- Enterprise Architecture (EA) team

ENTERPRISE ARCHITECTURE (EA) TEAM



Standardization



Accelerated Delivery



Transversal Automation



Self-Service & DevOps

The Enterprise Architecture team ensures the standardization and governance of technology, software design, and systems integration in alignment with business strategy enabling scalable, secure, and product-driven operations while supporting innovation and long-term planning.

AGENDA

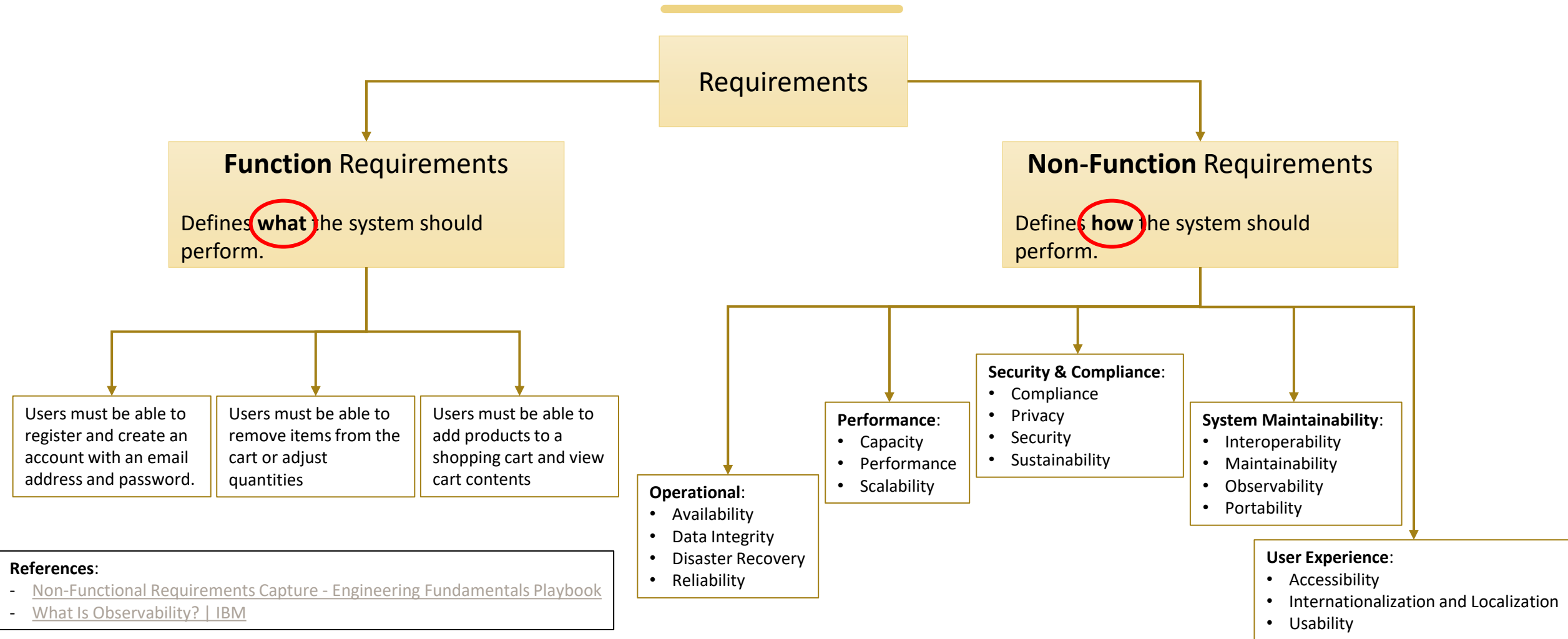
Theory:

- Functional Requirements VS Non-Functional Requirements
- Observability
 - Three pillars
 - Logs
 - Metrics
 - Traces
 - Example
 - Health Check
 - APM
 - Monitoring Tools
 - Application insights
 - DataDog

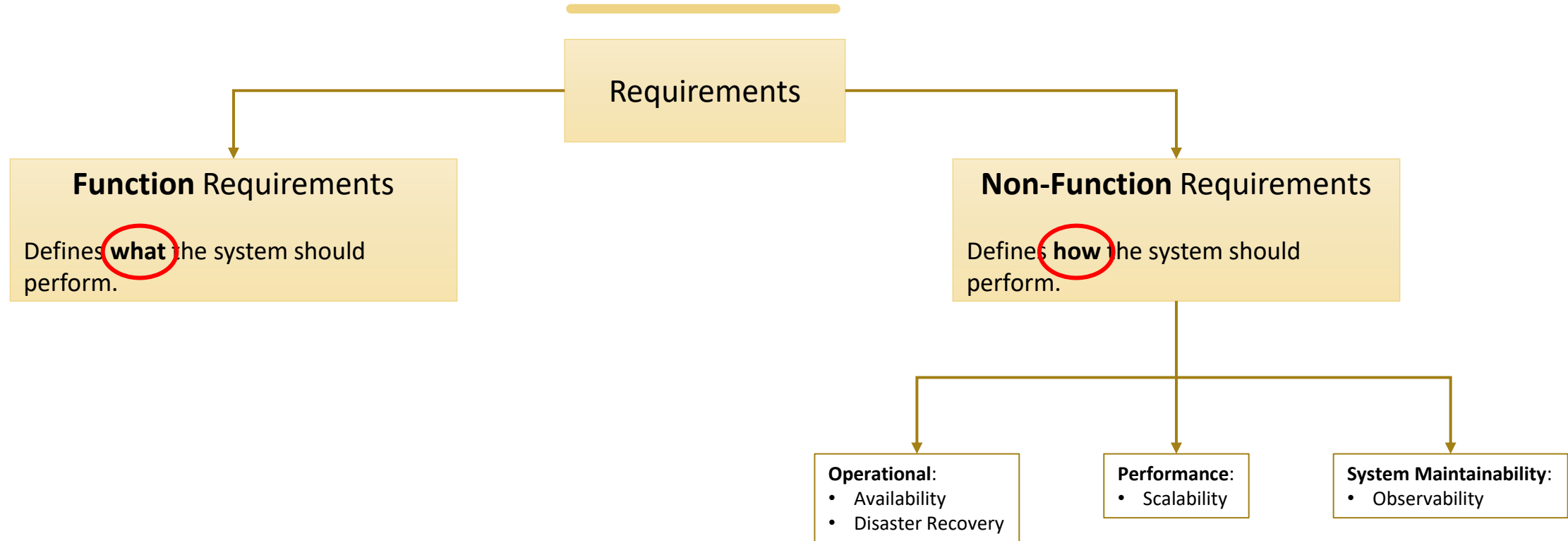
Implementation:

- Logs
 - General Concepts
 - Adding Log in Console App
 - Adding Log in ASP.NET
 - Configuring Log (via code or via configuration)
 - Structured Log
 - Serilog
 - Exercises
 - 1.1.Logging.Microsoft.SDK
 - 1.2.Logging.Microsoft.SDK.Structured
 - 1.3.Logging.Serilog
 - 1.4.Logging.Serilog.Datadog
- Metrics
- Traces
- Health Check
- Complete Exercise

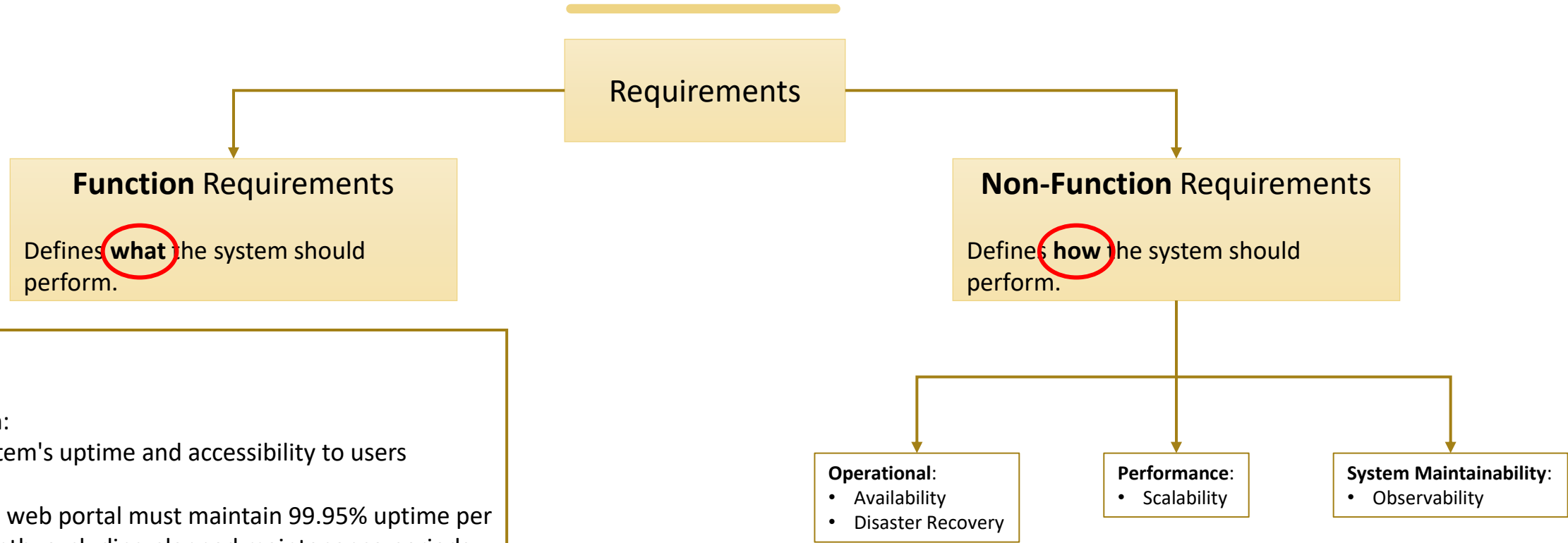
REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL



REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL



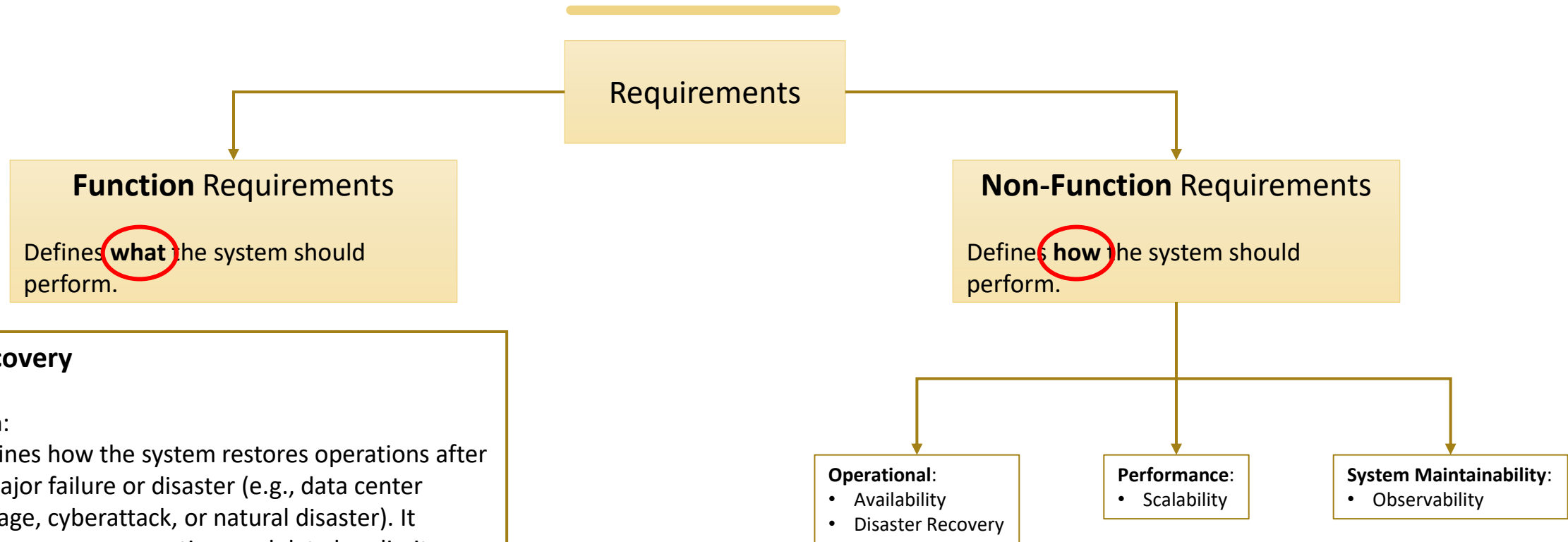
REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL



Availability

- **Definition:**
 - System's uptime and accessibility to users
- **Example:**
 - The web portal must maintain 99.95% uptime per month, excluding planned maintenance periods

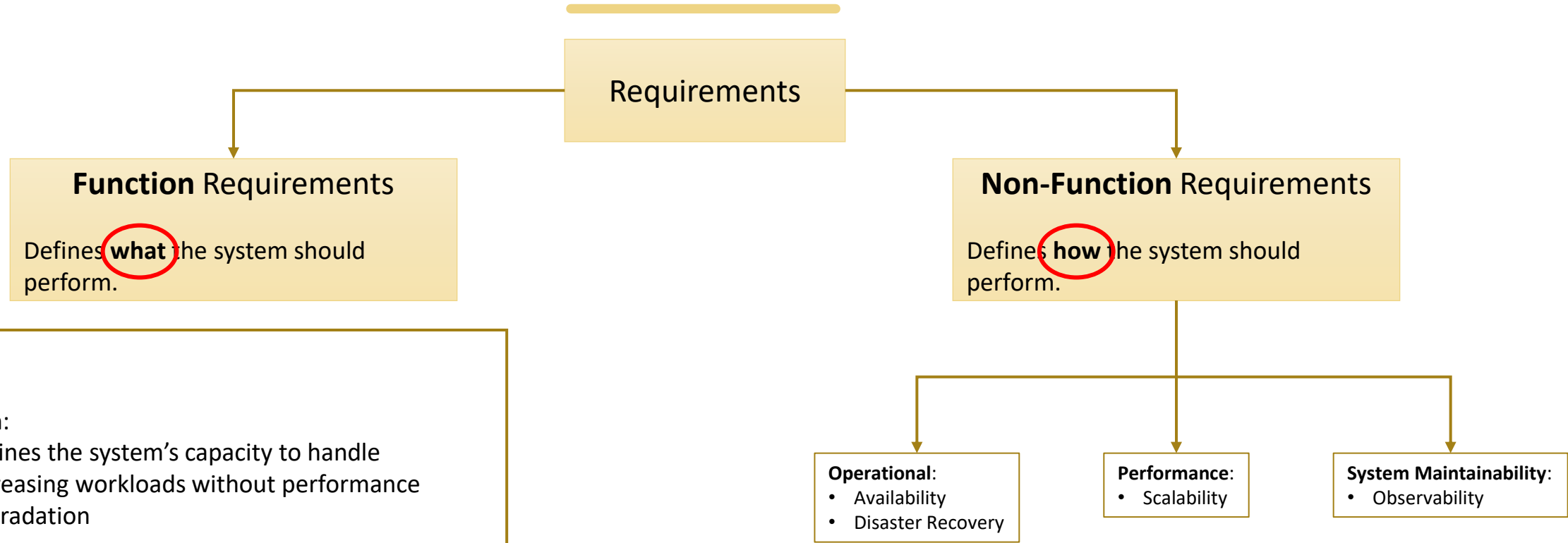
REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL



Disaster Recovery

- **Definition:**
 - Defines how the system restores operations after a major failure or disaster (e.g., data center outage, cyberattack, or natural disaster). It focuses on recovery time and data loss limits.
- **Example:**
 - In case of a system failure, services must be restored within 4 hours (**RTO**), and no more than 15 minutes of data (**RPO**) may be lost.

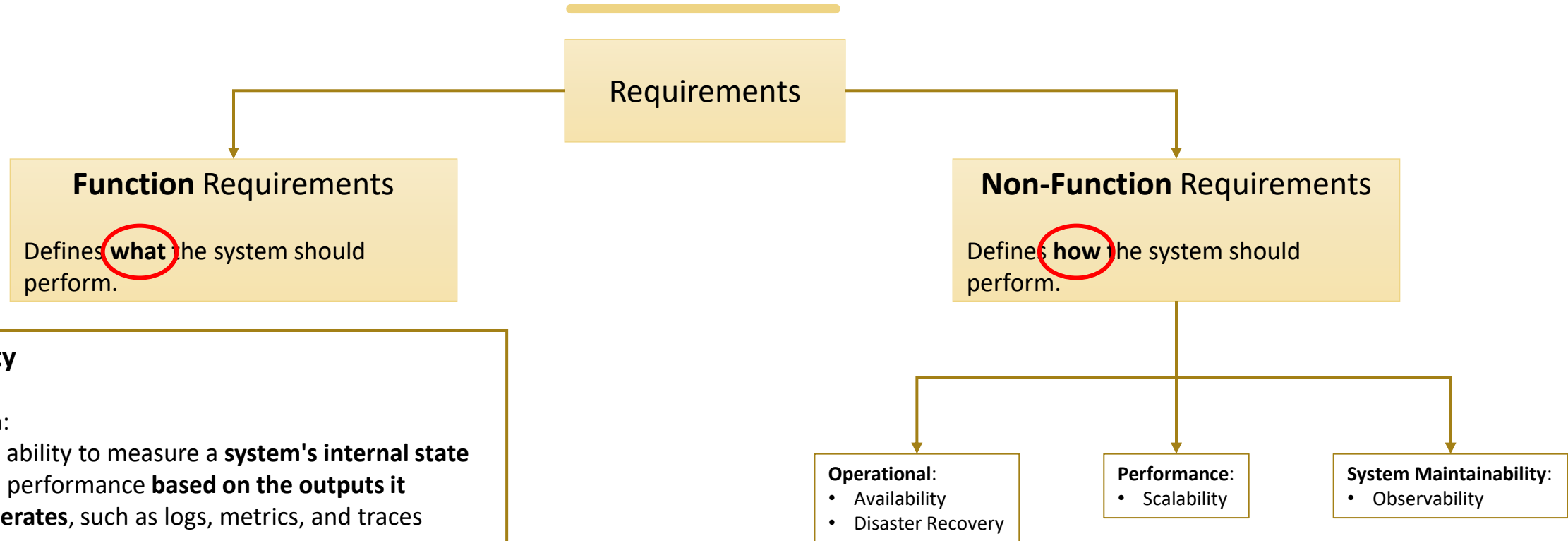
REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL



Scalability

- **Definition:**
 - Defines the system's capacity to handle increasing workloads without performance degradation
- **Example:**
 - The system must support a **10× increase** in active users **with less than 20% performance impact**.

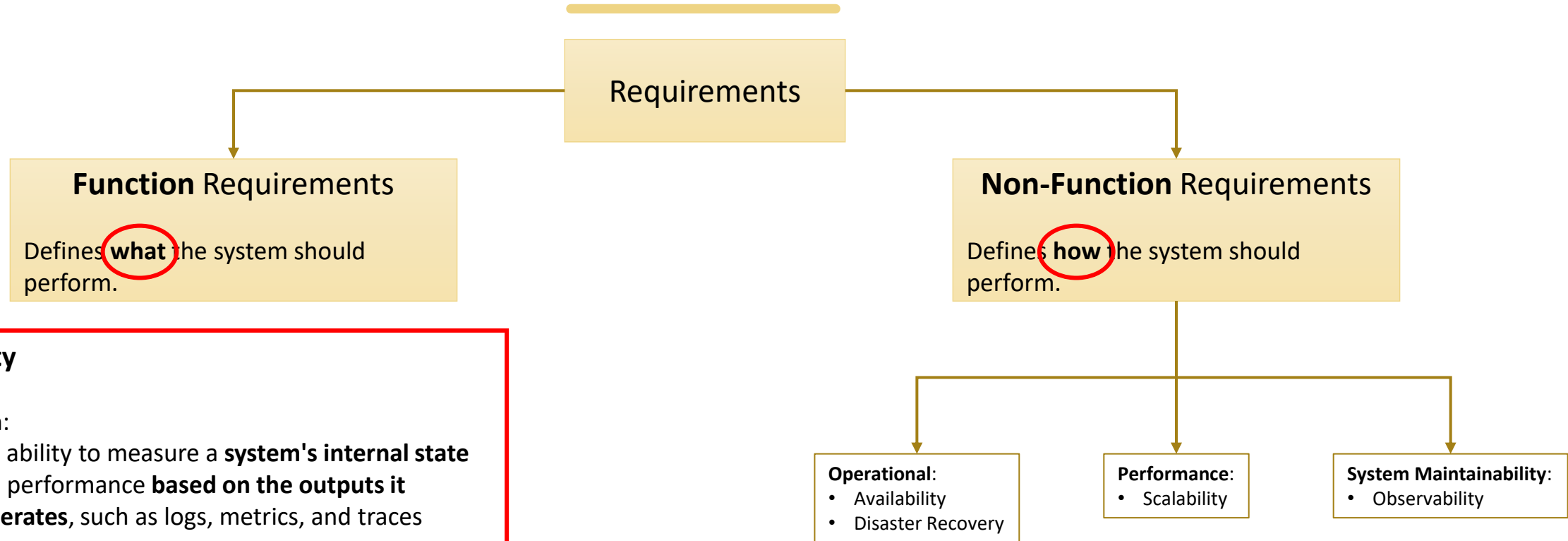
REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL



Observability

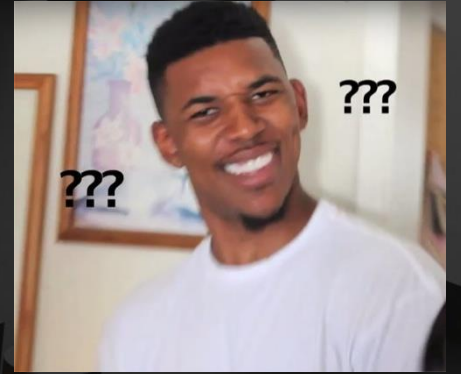
- **Definition:**
 - The ability to measure a **system's internal state** and performance **based on the outputs it generates**, such as logs, metrics, and traces
- **Example:**
 - The system must provide centralized logging, metrics, and distributed tracing that allow engineers to identify the root cause of incidents.

REQUIREMENTS – FUNCTIONAL VS NON-FUNCTIONAL

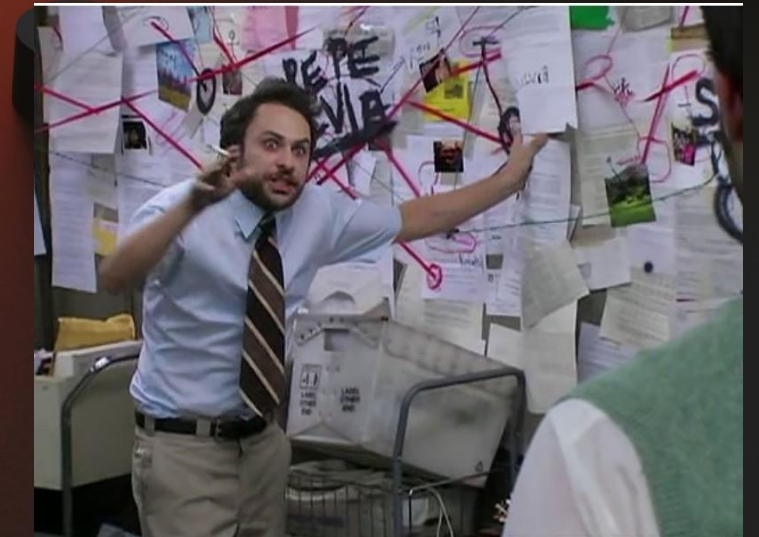


Observability

- **Definition:**
 - The ability to measure a **system's internal state** and performance **based on the outputs it generates**, such as logs, metrics, and traces
- **Example:**
 - The system must provide centralized logging, metrics, and distributed tracing that allow engineers to identify the root cause of incidents.



Q&A
Time



OBSERVABILITY

What:

- The ability to measure a system's **internal state** and performance **based on the outputs it generates**, such as logs, metrics, and traces.
- You can tell what's happening under the hood of the system without opening it.

Why:

- Provide **holistic view** of the **application health**.
- **Identify** and **diagnose failures** to get to the problem fast.

When:

- Always :)

PILLARS

Observability is composed of **three major pillars**:

- **Logs**

- **Logs** act as the system's “**captain's log**”, capturing significant events and what went wrong.



Logs

- **Metrics**

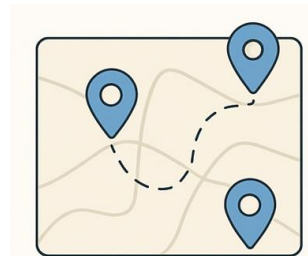
- Metrics are **numerical measurements** collected at regular intervals.



Metrics

- **Traces**

- Traces record the **entire journey of an operation** as it moves across multiple services.



Traces

LOGS

What:

- A log is a detailed **record of events** that occur in the system, with timestamps for each one.

Why:

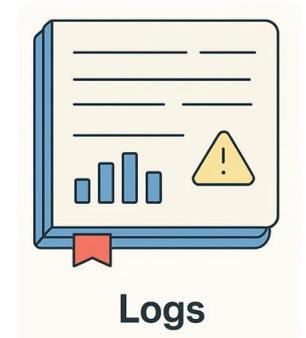
- To understand **what happened** and **why**
- Debugging & troubleshooting

When:

- When you need **context** around specific events, such as failures, exceptions, or unexpected behavior.
- Around key moments in your code

Example (file log “logs.txt”):

2025-11-03T14:12:45Z	INFO	AuthService	Login riuscito per utente: marco.rossi
2025-11-03T14:12:48Z	ERROR	AuthService	Login errato per utente: giuseppe.verdi



LOG LEVELS

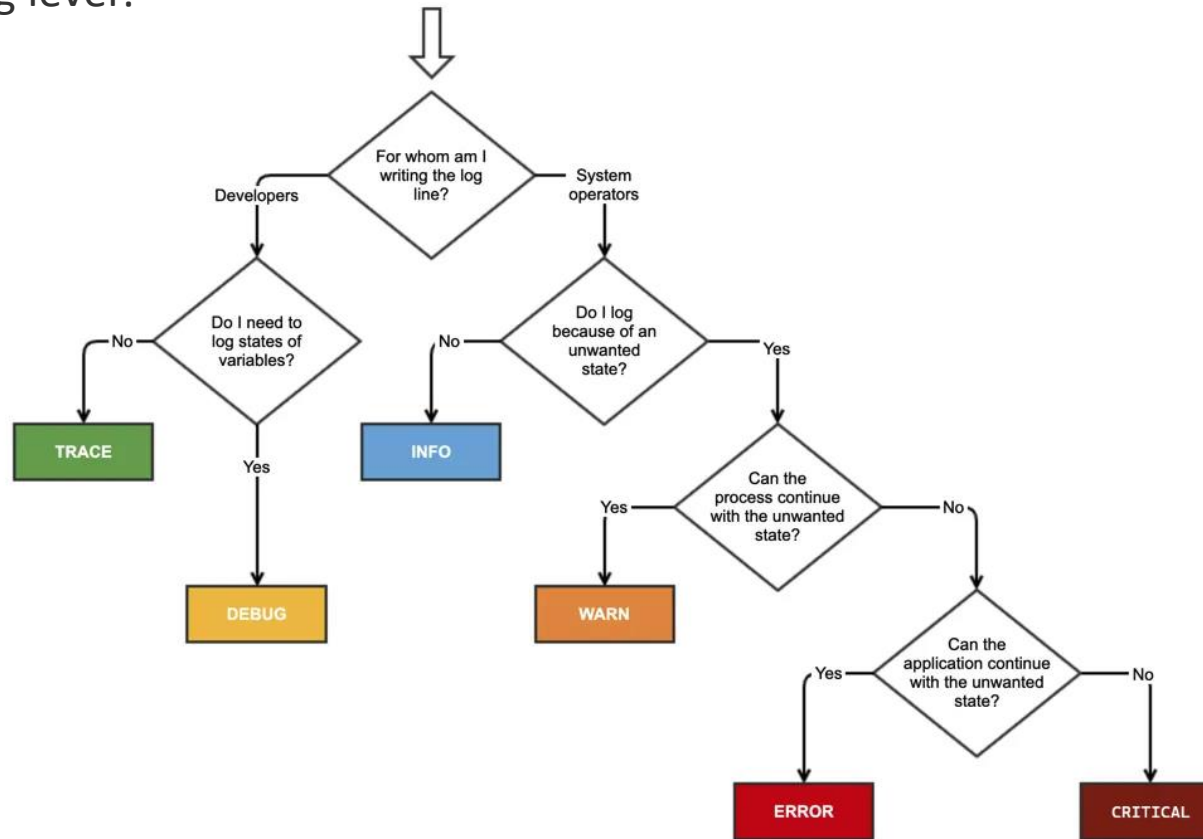
- Log levels are used to **categorize log events by severity** and control the verbosity of the logs

Name	Description
Trace	Logs that contain the most detailed messages. These messages may contain sensitive application data. These messages are disabled by default and should never be enabled in a production environment.
Debug	Logs that are used for interactive investigation during development. These logs should primarily contain information useful for debugging and have no long-term value.
Information	Logs that track the general flow of the application. These logs should have long-term value.
Warning	Logs that highlight an abnormal or unexpected event in the application flow, but do not otherwise cause the application execution to stop.
Error	Logs that highlight when the current flow of execution is stopped due to a failure. These should indicate a failure in the current activity, not an application-wide failure.
Critical	Logs that describe an unrecoverable application or system crash, or a catastrophic failure that requires immediate attention.
None	Not used for writing log messages. Specifies that a logging category should not write any messages.

[LogLevel Enum \(Microsoft.Extensions.Logging\) | Microsoft Learn](#)

LOG LEVELS

- How to choose the right log level?



Reference: [Step-by-Step Guide to Logs in C#: Best Practices and Tips](#) | by iamprovidence | Medium

STRUCTURED LOG

What:

- Structured Logs are **logs written in machine-readable format** (mostly in JSON), where the information is stored in **key-value pairs**.
- Structured logs are **queryable**.
- Example:

```
{  
  "timestamp": "2025-11-03T14:12:48Z",  
  "level": "ERROR",  
  "service": "PaymentService",  
  "event": "PaymentFailed",  
  "userId": 12345,  
  "orderId": 98765,  
  "error": "Timeout",  
  "gateway": "Stripe"  
}
```

Why:

- If the logs are written as unstructured plain text, it's hard to query them and extract useful information.
- Example:
 - 2025-11-03T14:12:48Z ERROR PaymentService – Payment failed for user 12345, order 98765, timeout connecting to gateway

STRUCTURED LOG

Unstructured logs:

```
2025-11-03T14:12:45Z INFO PaymentService - Starting payment process for user 12345, order 98765
2025-11-03T14:12:46Z INFO PaymentService - Sending payment request to gateway: Stripe
2025-11-03T14:12:48Z ERROR PaymentService - Payment failed for user 12345, order 98765, timeout connecting
to gateway
2025-11-03T14:13:12Z INFO PaymentService - Starting payment process for user 67890, order 98766
2025-11-03T14:13:13Z INFO PaymentService - Sending payment request to gateway: PayPal
2025-11-03T14:13:15Z INFO PaymentService - Payment successful for user 67890, order 98766, transaction ID
TX-PAYPAL-456
2025-11-03T14:14:02Z INFO PaymentService - Starting payment process for user 12345, order 98767
2025-11-03T14:14:05Z ERROR PaymentService - Payment failed for user 12345, order 98767, card declined
```

Example requirements: *"How many payments failed?"*

How can this requirement be fulfilled? Let's think it out together...

```
grep -E "ERROR PaymentService - Payment failed" logs.txt | wc -l
```

Find this hardcoded string
(if it changes or is slightly different...)

File to
search in

Command line
utility to count

STRUCTURED LOG

Structured logs:

```
{ "timestamp": "2025-11-03T14:12:45Z", "level": "INFO", "service": "PaymentService", "event": "PaymentStart", "userId": 12345, "orderId": 98765 }
{ "timestamp": "2025-11-03T14:12:46Z", "level": "INFO", "service": "PaymentService", "event": "PaymentRequestSent", "gateway": "Stripe", "orderId": 98765 }
{ "timestamp": "2025-11-03T14:12:48Z", "level": "ERROR", "service": "PaymentService", "event": "PaymentFailed", "userId": 12345, "orderId": 98765, "error": "Timeout", "gateway": "Stripe" }
{ "timestamp": "2025-11-03T14:13:12Z", "level": "INFO", "service": "PaymentService", "event": "PaymentStart", "userId": 67890, "orderId": 98766 }
{ "timestamp": "2025-11-03T14:13:13Z", "level": "INFO", "service": "PaymentService", "event": "PaymentRequestSent", "gateway": "PayPal", "orderId": 98766 }
{ "timestamp": "2025-11-03T14:13:15Z", "level": "INFO", "service": "PaymentService", "event": "PaymentSuccess", "userId": 67890, "orderId": 98766, "transactionId": "TX-PAYPAL-456" }
{ "timestamp": "2025-11-03T14:14:02Z", "level": "INFO", "service": "PaymentService", "event": "PaymentStart", "userId": 12345, "orderId": 98767 }
{ "timestamp": "2025-11-03T14:14:05Z", "level": "ERROR", "service": "PaymentService", "event": "PaymentFailed", "userId": 12345, "orderId": 98767, "error": "CardDeclined", "gateway": "Stripe" }
```

Example requirements: *"How many payments failed?"*

"How many payments failed?"

```
PaymentLogs
| where service == "PaymentService"
| where event == "PaymentFailed"
| summarize FailedPayments = count()
```

"How many payments failed today?"

```
PaymentLogs
| where service == "PaymentService"
| where event == "PaymentFailed"
| where timestamp between (startofday(now())) .. now()
| summarize FailedPayments = count()
```

"Failures Over Time (Trend binned by 1h)"

```
PaymentLogs
| where service == "PaymentService"
| where event == "PaymentFailed"
| summarize Failures = count() by bin(timestamp, 1h)
```

ACH:

To perform queries over logs, you must have a "Monitoring Tool" (we'll see this later...)

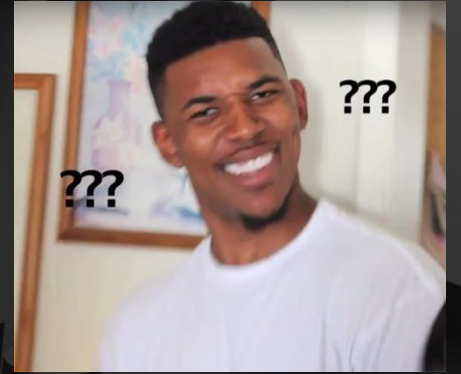
STRUCTURED LOG (COMPARISON)

Dimension	Unstructured Log	Structured Log
Format	Plain text	Key-value pairs (usually JSON)
Readability	Easy to read by humans Hard to read by machines	Easy to read by machines Hard to read by humans
Searchability	Low	High
Data Extraction	Low	High

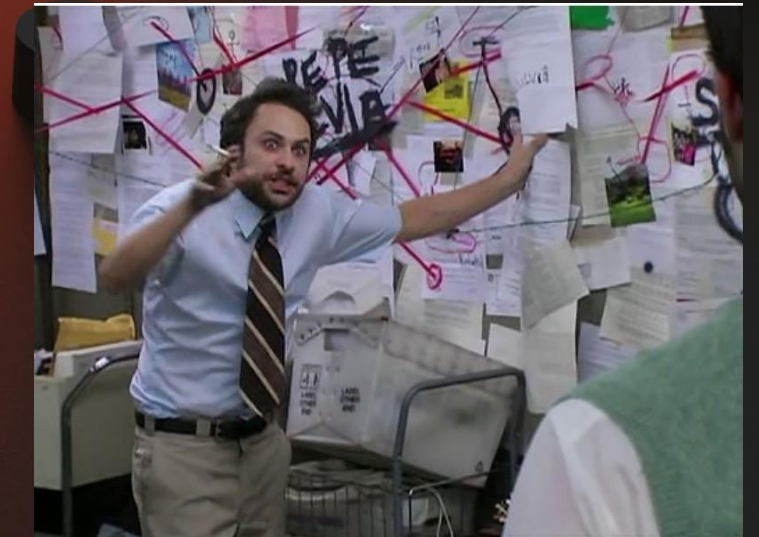
LOG TYPES

If even there is not official standard that defines log categories:

Log Type	Description	Typical Use	Who Uses it
Technical Logs (Application Logs)	Events generated by the application code	Debugging, troubleshooting, performance analysis.	Developers
System Logs	Events generated by the Operating System (OS) or the infrastructure	Monitoring infrastructure health, uptime, system errors.	IT Operations
Audit Logs	Record who (e.g., users) did what , when , and how in the system.	Compliance, traceability, accountability. E.g., my system must be compliant with an ISO standard.	Compliance officers
Security Logs	Authentication, authorization, intrusion detection, firewalls.	Security monitoring.	Security Team (SOC)
Business Logs	Domain-specific actions like order creation, invoice sent, shipment updated. These logs only make sense within the context of the application domain .	Business intelligence, user behavior, KPIs.	Product Owners / Business Analyst



Q&A
Time



METRICS

What:

- Metrics are **numerical measurements** collected at **regular intervals**.



Why:

- To **monitor trends** and **detect anomalies** over time

When:

- When you need to **track performance continuously** and **trigger alerts** on threshold.

METRICS



	Technical	
What		
Why		
When		
Example		

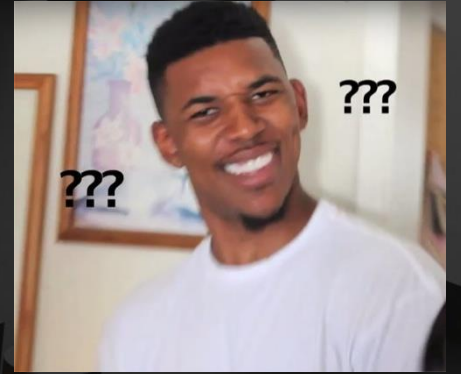


METRICS

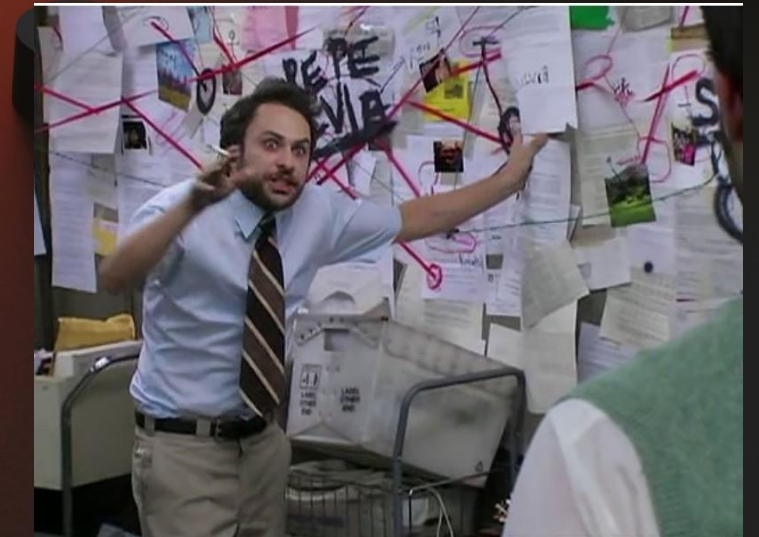
	Technical	
What	Numeric measurements collected at regular intervals to represent <u>system performance</u> .	
Why	To monitor trends and detect anomalies over time	
When	To track performance continuously and trigger alerts on threshold breaches	
Example	• API response time = 420 ms • Error rate (PaymentService) = 3.2 %	

METRICS

	Technical	Business
What	Numeric measurements collected at regular intervals to represent <u>system performance</u> .	Numeric measurements collected at regular intervals to represent <u>business outcomes</u> .
Why	To monitor trends and detect anomalies over time	To understand how the system supports business goals .
When	To track performance continuously and trigger alerts on threshold breaches	To extract business outcomes by product owners and business analysts .
Example	• API response time = 420 ms • Error rate (PaymentService) = 3.2 %	• Number of completed orders per hour = 250 • Cart abandonment rate = 8 %



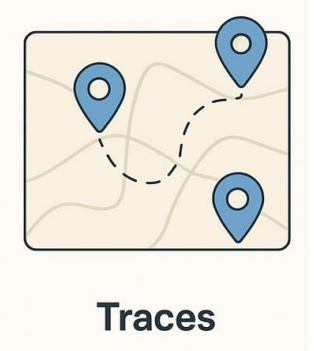
Q&A
Time



TRACES

What:

- A **trace** is the **entire journey of an operation** as it moves across multiple services.
 - An “operation” may refer to **a run for background job systems** (system always running that performs the same operations indefinitely)
 - An “operation” may refer to **a HTTP Request for Web API**



Why:

- To investigate the entire journey of the operation.

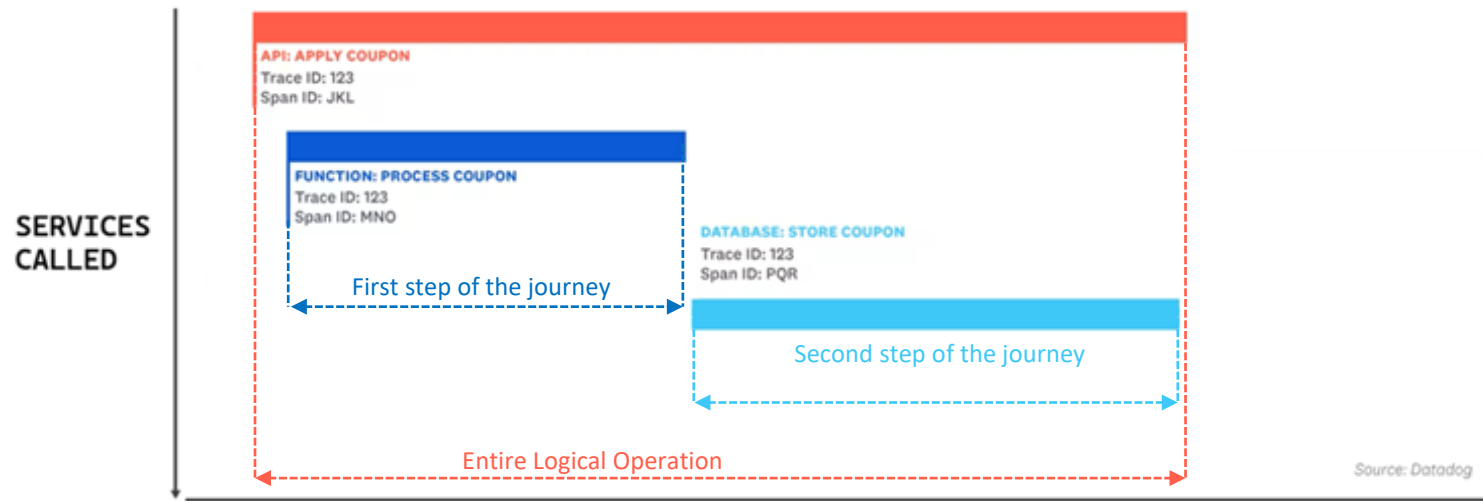
When:

- When you need to focus on **the entire end-to-end analysis of the operation**.

TRACES

Example:

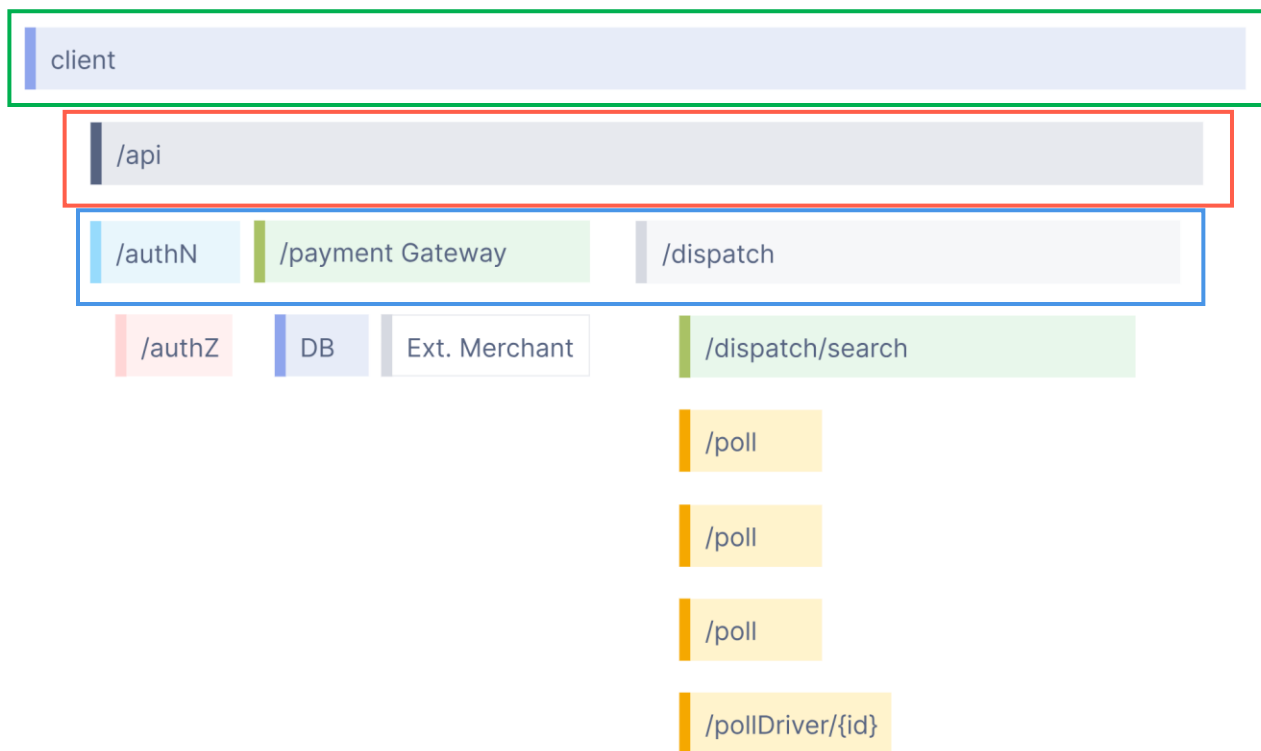
- An e-shop Web API that apply a discount coupon
- To achieve this **Apply Coupon operation**, the system needs to:
 - **Verify the coupon** (e.g., well-formatted with regex)
 - **Store the coupon in the database** (to be not reusable)



TRACES - GLOSSARY

- **Trace:**
 - It represents the entire journey of an operation as it moves across multiple services.
 - A trace it's made up of **multiple spans**.
- **Span:**
 - A **single unit of work** within a trace. Such as a function call, database query, or external API call.
 - There is a parent-child relationship between spans:
 - **Parent Span:** the operation that **triggers** other work (e.g., a controller calling the database)
 - **Child Span:** the operation **triggered** by the parent (Database operation)
- **ID:**
 - **TraceId:** Unique identifier associated to the trace.
 - **SpanId:** Unique identifier associated to a single span within a trace.
- **Context Span:**
 - **Every span contains an object called Context**, that is a **bag of information to understand the position of the span inside the trace**
 - The Context object usually contains 3 main attributes:
 - **TraceId:** Identifies the overall trace and it's the same for all the spans inside a trace
 - **SpanId:** Identifies this specific span
 - **ParentSpanId:** Identifies the span that triggered this specific span

TRACES – HOW TO READ THE FLAME GRAPH

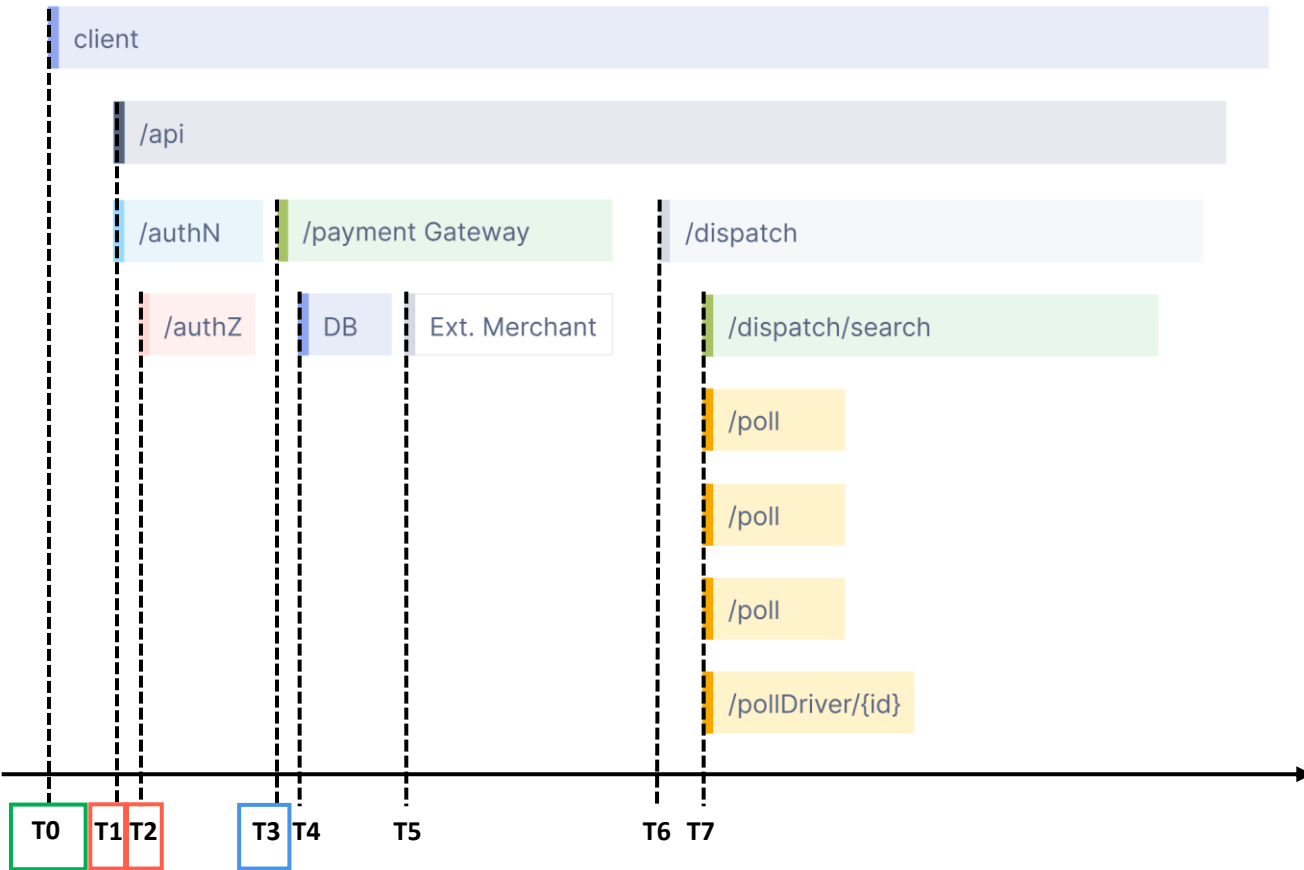


Vertical axis (parent-child relationship):

- Flame Graph shows the parent-child relationship between spans.
- E.g.:
 - The first span that covers the entire operation is called **“client”**
 - The **“client”** span triggers the **“/api”** span
 - The **“/api”** span triggers:
 - **“/authN”** span
 - That triggers the **“/authZ”** span
 - **“/payment Gateway”**
 - That triggers the **“DB”** and **“Ext. Merchant”** spans
 - **“/dispatch”**

[Observability primer](#) | [OpenTelemetry](#)

TRACES – HOW TO READ THE FLAME GRAPH



Horizontal axis (temporal time):

- It show the time when every span is executed

• E.g.:

- **T0: “client”** span started
- Simultaneously
 - T1: “/api” span started
 - T1: “/authN” span started
- **T2: “/authZ”** span started
- **T3: “/payment Gateway”** span started
- **T4: “DB”** span started
- **T5: “Ext. Merchant”** span started
- **T6: “/dispatch”** span started
- Simultaneously
 - **T7: “/dispatch/search”** span started
 - **T7: “/poll”** span started
 - **T7: “/poll”** span started
 - **T7: “/poll”** span started
 - **T7: “/pollDriver/{Id}”** span started

DISTRIBUTED TRACING

Distributed system:

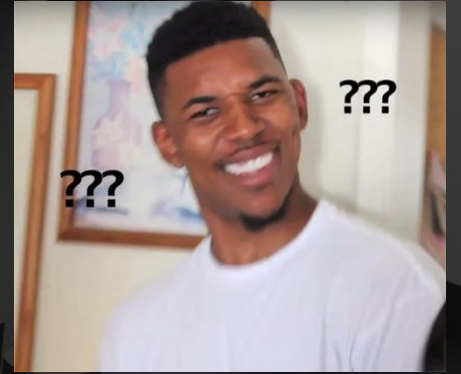
- A distributed system is a **collection of independent components**, often running on different machines, **that** communicate and **coordinate with each other to accomplish a single task**.

Example:

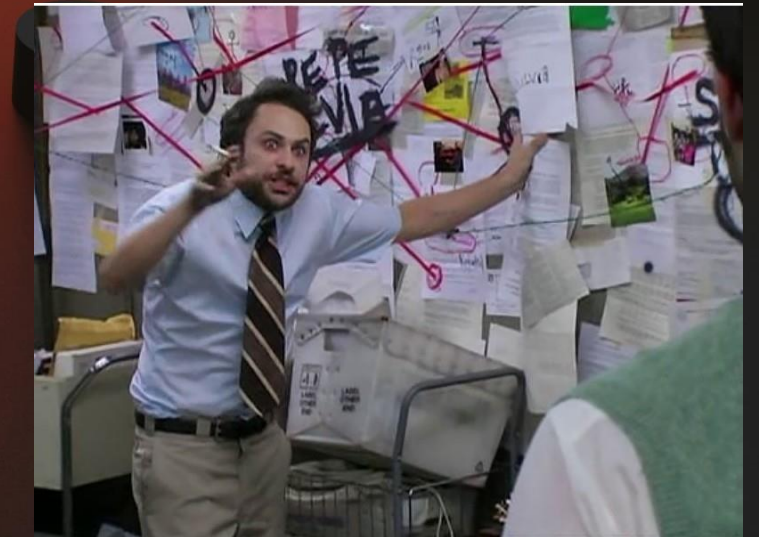
- When a customer places an order in an e-commerce site.
 - The **Frontend service** receives the request and validates the input.
 - The **Order service** creates the order record in the **database**.
 - The **Payment service** processes the payment and updates the **database**.
 - The **Email service** sends the confirmation.

Distributed Tracing:

- **Distributed Tracing = Tracing in Distributed Systems**
- Distributed tracing is the **entire journey of an operation** as it moves across multiple services of a distributed system.



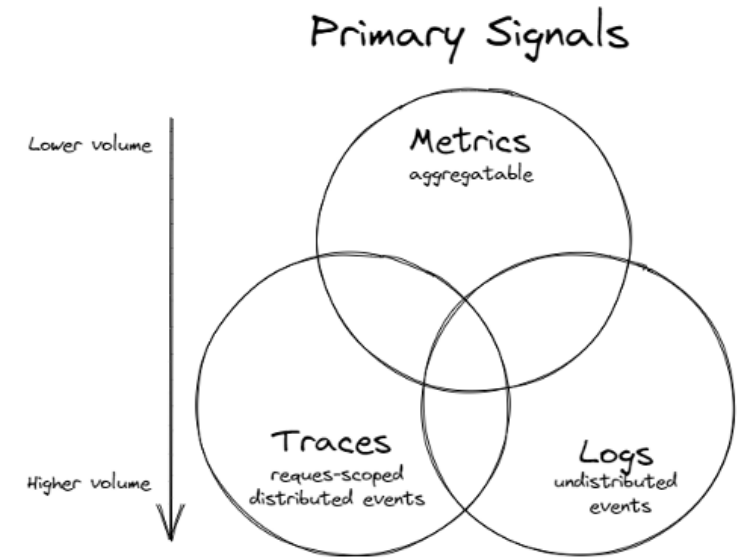
Q&A
Time



ALL TOGETHER: LOGS + METRICS + TRACES

When to use the right tool:

- **Logs:**
 - Use logs to **track detailed information about an event**, particularly errors, warnings or other exceptional situations.
- **Metrics:**
 - Use metrics to track the occurrence of an event, counting of items.
- **Traces:**
 - A trace provides visibility into how a request is processed across multiple services.



[Logs vs Metrics vs Traces - Engineering Fundamentals Playbook](#)

ALL TOGETHER: LOGS + METRICS + TRACES

Let's see the three pillars in a practical scenario.

- **Scenario Description:**
 - The process of buying an item on an e-shop system involves several components:
 1. **Frontend Service:** receives the request and validates order data
 2. **Order Service:** creates the order in the database
 3. **Payment Service:** calls an external provider to process the payment
 4. **Notification Service:** sends the confirmation email
 - Within a day, **multiple orders** are processed, **some succeed, some fail** due to different issues.

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO OrderService Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO OrderService Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO NotificationService Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO OrderService Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO PaymentService Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO PaymentService Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO OrderService Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO NotificationService Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO OrderService Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO OrderService Updating order 98767 status to PaymentFailed
```

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO OrderService Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO OrderService Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO NotificationService Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO OrderService Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO PaymentService Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO PaymentService Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO OrderService Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO NotificationService Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO OrderService Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO OrderService Updating order 98767 status to PaymentFailed
```

Metrics:

payments_total:	3
payments_successful:	1
payments_failed:	2
payment_failure_rate:	66,7%
Payment_avg_duration:	2s
email_notifications_sent:	2

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO OrderService Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO OrderService Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO NotificationService Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO OrderService Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO PaymentService Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO PaymentService Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO OrderService Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO NotificationService Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO OrderService Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO OrderService Updating order 98767 status to PaymentFailed
```

Metrics:

payments_total:	3
payments_successful:	1
payments_failed:	2
payment_failure_rate:	66,7%
Payment_avg_duration:	2s
email_notifications_sent:	2

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO OrderService Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO OrderService Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO NotificationService Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO OrderService Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO PaymentService Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO PaymentService Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO OrderService Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO NotificationService Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO OrderService Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO OrderService Updating order 98767 status to PaymentFailed
```

Metrics:

payments_total:	3
payments_successful:	1
payments_failed:	2
payment_failure_rate:	66,7%
Payment_avg_duration:	2s
email_notifications_sent:	2

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO OrderService Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO OrderService Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO NotificationService Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO OrderService Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO PaymentService Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO PaymentService Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO OrderService Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO NotificationService Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO OrderService Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO OrderService Updating order 98767 status to PaymentFailed
```

Metrics:

payments_total:	3
payments_successful:	1
payments_failed:	2
payment_failure_rate:	66,7%
Payment_avg_duration:	2s
email_notifications_sent:	2

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO OrderService Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO OrderService Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO NotificationService Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO OrderService Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO PaymentService Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO PaymentService Payment successful for order 98766, transactionId tx-PAYPAL-456
2025-11-03T09:15:17Z INFO OrderService Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO NotificationService Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO OrderService Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO OrderService Updating order 98767 status to PaymentFailed
```

Metrics:

payments_total:	3
payments_successful:	1
payments_failed:	2
payment_failure_rate:	66,7%
Payment_avg_duration:	2s
email_notifications_sent:	2

ALL TOGETHER: LOGS + METRICS + TRACES

Logs:

```
2025-11-03T09:00:00Z INFO FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO OrderService Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService Payment failed for order 98765, user 12345, timeout connecting to gateway
2025-11-03T09:00:05Z INFO OrderService Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO NotificationService Sending payment failure email to user 12345

2025-11-03T09:15:12Z INFO FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO OrderService Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO PaymentService Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO PaymentService Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO OrderService Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO NotificationService Sending confirmation email to user 67890

2025-11-03T10:42:33Z INFO FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO OrderService Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO OrderService Updating order 98767 status to PaymentFailed
```

Metrics:

payments_total:	3
payments_successful:	1
payments_failed:	2
payment_failure_rate:	66,7%
Payment_avg_duration:	2s
email_notifications_sent:	2

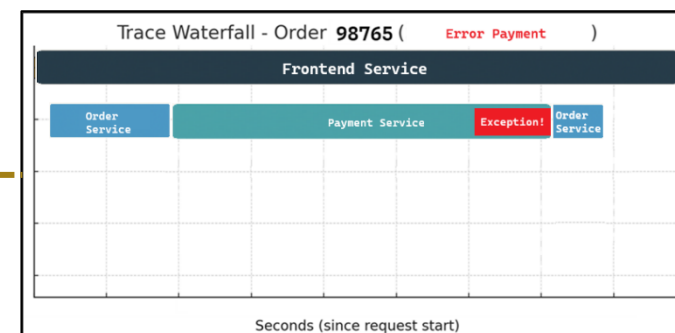
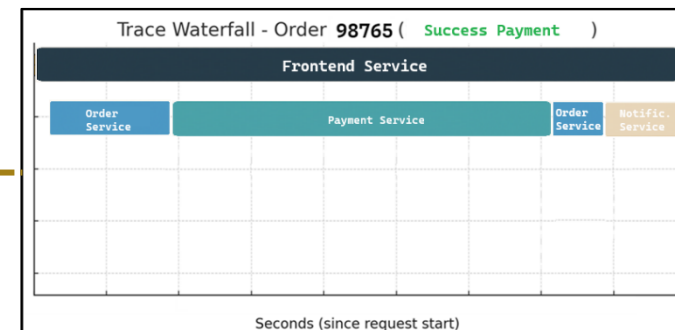
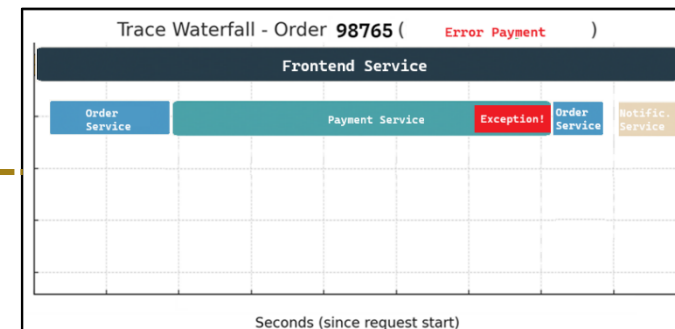
ALL TOGETHER: LOGS + METRICS + TRACES

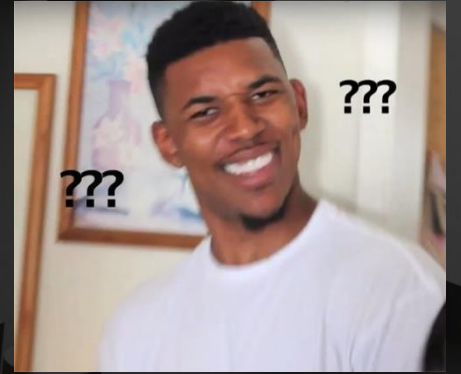
Logs:

```
2025-11-03T09:00:00Z INFO FrontendService User 12345 started checkout for cart 56789
2025-11-03T09:00:01Z INFO OrderService Creating order record for user 12345, orderId 98765
2025-11-03T09:00:02Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T09:00:04Z ERROR PaymentService Payment failed for order 98765, user 12345, timeout
connecting to gateway
2025-11-03T09:00:05Z INFO OrderService Updating order 98765 status to PaymentFailed
2025-11-03T09:00:06Z INFO NotificationService Sending payment failure email to user 12345
```

```
2025-11-03T09:15:12Z INFO FrontendService User 67890 started checkout for cart 56800
2025-11-03T09:15:13Z INFO OrderService Creating order record for user 67890, orderId 98766
2025-11-03T09:15:14Z INFO PaymentService Initiating payment via gateway: PayPal
2025-11-03T09:15:16Z INFO PaymentService Payment successful for order 98766, transactionId TX-PAYPAL-456
2025-11-03T09:15:17Z INFO OrderService Updating order 98766 status to Paid
2025-11-03T09:15:18Z INFO NotificationService Sending confirmation email to user 67890
```

```
2025-11-03T10:42:33Z INFO FrontendService User 22222 started checkout for cart 56900
2025-11-03T10:42:34Z INFO OrderService Creating order record for user 22222, orderId 98767
2025-11-03T10:42:35Z INFO PaymentService Initiating payment via gateway: Stripe
2025-11-03T10:42:36Z ERROR PaymentService Payment failed for order 98767, user 22222, card declined
2025-11-03T10:42:37Z INFO OrderService Updating order 98767 status to PaymentFailed
```





Q&A
Time

